

Module 7 — Household & User Management

(modules/household)

Purpose

Provide the simplest possible multi-user foundation that every other module depends on: who is logged in, what household they belong to, and which **Entity** a record belongs to. An **Entity** is simply a named owner made up of one or more members — three individuals (A, B, C) plus a joint entity for the couple (A+B) is the expected shape.

Actors & User Stories

- *As the household owner*, I create accounts for the other members of my household directly — no email-invite flow needed on a private server.
- *As a member*, I log in and see the whole household's finances, and can switch the view to any single entity (mine, my partner's, ours) to see just that slice.
- *As a viewer* (e.g. a trusted read-only guest), I can look at summaries but can't edit anything.
- *As a parent*, I track a dependent child's expenses under their own profile even though they never log in.

Data Model

- **User**: `id`, `email`, `display_name`, `password_hash`, `role(OWNER|MEMBER|VIEWER)`, `is_dependent`, `is_deleted`, `locale`, `timezone`, `created_at`, `updated_at`.
`is_dependent=true` users have `password_hash = NULL` and never log in.
- **Household**: `id`, `name`, `created_by`, `created_at`. One household per deployment; multi-household support is explicitly out of scope.
- **HouseholdMember**: `id`, `household_id`, `user_id`, `role(OWNER|MEMBER|VIEWER)`, `joined_at`, `left_at?`. `role` is the single source of truth for RBAC — there is no separate per-module override table.
- **Entity**: `id`, `household_id`, `name`, `member_ids (UUID[])`, `is_deleted`, `created_at`. Every financial record in every module carries an `entity_id` FK to this table. There is **no type enum** — an entity with one member is an individual, an entity with several is a joint entity (e.g. the couple). Nothing in the system branches on entity "kind".
- **Session**: `id`, `user_id`, `created_at`, `expires_at`, `entity_id` (persisted "viewing as" filter). Server-side session (Redis-backed), httpOnly `SameSite=Lax` cookie, CSRF token on state-changing requests.

That's the whole data model — five tables. No **PermissionOverride**, **EntityReattribution**, **InvitePending**, or GDPR data-subject-request tables. Re-attributing a record to a different entity is a normal field edit on that record (module-specific), guarded by the same RBAC check and recorded in **AuditLog** (§1a) — not a separate approval workflow.

Functional Requirements

- **FR-7.1 Entity Attribution.** Every record in every module carries `entity_id`. An entity is a named owner with one or more members; individual and joint entities are the same thing with a different member count. Entity is an **attribution and filtering** dimension, **not a security boundary** — see FR-7.3.
- **FR-7.2 Entity Selector.** A top-bar dropdown filters the whole app to one entity, or to "todas". Persisted in `Session.entity_id`; defaults to whatever was selected last, "todas" on first login.
- **FR-7.3 Role-Based Access Control (3 roles, no per-entity isolation).** Every logged-in household member can **read every entity's data** — this is a 4-person household that already shares a bank account, and per-entity read isolation buys nothing but bugs. Roles differ only in write power: OWNER: full control, can create/edit/remove users and entities. MEMBER: read everything, write everything. VIEWER: read-only everywhere. Enforced by one middleware check per request on `user_role`. (If real isolation is ever needed, it is added here and only here — every module already carries `entity_id`.)
- **FR-7.4 Member Management.** OWNER creates a `User` directly (email + temporary password the member changes on first login) — no invitation/token/email flow needed for a private single-household deployment.
- **FR-7.5 Dependent Profiles.** `is_dependent=true`: no password, no session, no login; exists purely so a parent can attribute expenses to that profile. Appears in entity pickers.
- **FR-7.6 Member Departure.** Soft-delete via `left_at`; sessions revoked; entities where they were the only member become read-only. Multi-member entities are unaffected and keep functioning (just flagged in the UI as "includes a departed member").
- **FR-7.7 Entity Auto-Creation.** On member creation, one single-member entity is auto-created for them. Multi-member entities (e.g. the couple) are created manually by the OWNER when needed — there is no automatic "couple detection" and no mandatory all-members entity.
- **FR-7.8 Session & CSRF.** Server-side session (Redis), httpOnly `SameSite=Lax` cookie, double-submit CSRF token on state-changing requests; session invalidated on logout and on password change.

Automation Rules

- Auto-create one single-member entity per new `User`.
- Module queries filter by `entity_id` only when the user has selected an entity in the selector; no visibility computation is involved.
- Member departure: revoke active sessions; flip any entity they were the sole member of to read-only.

UI / Screens

- **Members:** a simple list (name, email, role, joined date, active/departed) with an "Add member" form (email, display name, role, temporary password) and an "Add dependent" form (display name only).
- **Entity Selector** (global top bar): dropdown of every entity plus "todas"; persists selection.
- **My Account:** change own password, display name, locale, timezone.

Security & RBAC

- **OWNER:** everything MEMBER can do, plus create/remove members, create entities, change roles.
- **MEMBER:** read and write across all entities in the household.
- **VIEWER:** read-only everywhere; no CRUD.
- **Data isolation:** none *within* the household — by explicit decision. The security boundary is authentication: an unauthenticated request sees nothing, and there is exactly one household per deployment.

API Surface

- `POST /auth/login` · `POST /auth/logout` · `GET /auth/me`.
- `GET /households/{id}/members` · `POST /households/{id}/members` (OWNER creates a member or dependent directly) · `PATCH /members/{id}` · `DELETE /members/{id}` (soft-delete).
- `GET /entities` · `POST /entities` (OWNER; multi-member entities — single-member ones are auto-created) · `PATCH /entities/{id}`.
- `GET /sessions/current` · `POST /sessions/entity` (switch the active entity filter, returns a fresh CSRF token).

Edge Cases & Validation

- **Dependent without login:** no `password_hash`, no `Session` row; only an OWNER/MEMBER can create expenses attributed to them.
- **Member departure:** `left_at` set; entities where they were the sole member → read-only; multi-member entities keep functioning, unaffected.
- **Re-attributing a record to a different entity:** a normal field edit on that record, guarded by the same RBAC check, recorded in `AuditLog` (before/after `entity_id`) — no separate reattribution workflow or approval step.
- **Child ageing into login:** OWNER flips `is_dependent=false` and sets a password; the existing entity and its history are preserved unchanged.
- **Last OWNER safeguard:** the system must always have at least one active OWNER; block removing or demoting the only remaining OWNER.

Open Questions / Decisions

#	Question	Recommended Default
1	Multi-household support?	Out of scope — one household per deployment.
2	Formal invitation/email flow?	No — OWNER creates accounts directly with a temporary password.
3	Per-module permission overrides beyond the 3 roles?	No — OWNER/MEMBER/VIEWER is sufficient; revisit only if a real need emerges.
4	Formal GDPR export/delete tooling?	Out of scope for a private single-household server; soft-delete + <code>AuditLog</code> is sufficient.
5	Session timeout?	30 days, extended on activity.
6	Per-entity read isolation between household members?	No — everyone reads everything; entity is attribution + a filter, not a permission. Revisit only if a member actually asks for privacy.
7	<code>Entity.type</code> (INDIVIDUAL/JOINT/HOUSEHOLD)?	Dropped — derivable from <code>member_ids</code> length, and nothing branches on it.

Definition of Done

- User/HouseholdMember/Entity CRUD tested (unit + integration).

- Member creation (incl. dependent) and departure (soft-delete) work end-to-end.
- RBAC middleware enforced on every endpoint: writes require OWNER/MEMBER, VIEWER is read-only.
- Entity selector persists across page loads and correctly filters (including "todas").
- Session + CSRF: login, logout, and password-change all correctly invalidate/rotate sessions.
- Last-OWNER safeguard tested.
- Seed: one household with three members (two adults + one dependent child) and one joint entity for the couple.
- Docs: how login/session/CSRF works, recorded in [docs/architecture.md](#) or [docs/decisions/](#).

Integration Contract

- **Exposes:** `Entity` (id, type, name, member_ids[]) and `HouseholdMember.role` to every module; RBAC context (`user_role`, accessible `entity_ids`) as request-scoped middleware state; `Session` (`user_id`, `entity_id`, `csrf_token`).
- **Consumes:** nothing — foundational, built first, in Phase 0.
- **Guarantees:** no record is ever served or written unless the requester's role grants access to that record's `entity_id`.